

FT_IRC – SOCKETS, POLL() AND EVALUATION SURVIVAL GUIDE

PART 1 – WHAT IS A SOCKET

A socket is an endpoint for network communication. Your program talks to the outside world through a "file descriptor" (an int) that the OS treats almost like a file, but bytes written to it travel over the network instead of to disk.

For a TCP server (which is what ft_irc is), the lifecycle is:

```
socket()    -> create the endpoint, get back an fd
setsockopt  -> configure options on it (SO_REUSEADDR)
bind()      -> attach it to a local address+port ("listen on 0.0.0.0:6667")
listen()    -> mark it as a socket that accepts incoming connections
accept()    -> pull ONE pending connection off the queue, get a NEW fd
               for that specific client (the original fd stays "listening")
recv()/send() -> read/write bytes on a CLIENT fd
close()     -> release the fd
```

This is exactly what config_socket() + accept_new_client() do in server.cpp:

```
fd_serv = socket(AF_INET, SOCK_STREAM, 0);    // TCP socket
setsockopt(fd_serv, SOL_SOCKET, SO_REUSEADDR, ...); // reuse port
                                                    // immediately
                                                    // after restart

bind(fd_serv, &addr, sizeof(addr));           // attach to the port
listen(fd_serv, SOMAXCONN);                   // start queueing conns

// later, in the loop:
int client_fd = accept(fd_serv, NULL, NULL); // one NEW fd per client
```

Key point for the evaluation: fd_serv (the listening socket) NEVER sends or receives IRC data. It only produces new client fds via accept(). All actual chat traffic happens on the per-client fds stored in poll_fds / clients.

SOCK_STREAM = TCP: a reliable, ordered BYTE STREAM. There is no concept of "messages" at the TCP level – if you send "PRIVMSG foo\r\n" and then "PRIVMSG bar\r\n", the receiver might get it as one recv() call containing both, or split across two, or even split mid-line. This is WHY client.cpp buffers input and looks for '\n' to find line boundaries (is_line_complete/extract_line). This is one of the most commonly asked evaluation questions: "what happens if a client sends half a line?" – answer: it stays in the buffer until the rest arrives.

```

49 BLOCKING VS NON-BLOCKING
50 -----
51 By default, recv()/accept()/send() BLOCK: the calling thread freezes
52 until data is available / a connection arrives / buffer space frees up.
53 That's fine for a program handling ONE client, but ft_irc must handle
54 MANY clients at once with a SINGLE thread – so blocking on any one fd
55 would freeze the whole server for everyone else.
56
57 The fix: fcntl(fd, F_SETFL, O_NONBLOCK) makes calls return immediately
58 with -1 (errno EAGAIN/EWOULDBLOCK) instead of blocking when there's
59 nothing to do. This is done for every fd right after it's created:
60
61     set_non_block(fd_serv)    inside config_socket()
62     set_non_block(client_fd)  inside accept_new_client()
63
64 But if every call is non-blocking, how does the server know WHEN there's
65 actually something to read/write, without spinning in a busy loop
66 wasting 100% CPU checking every fd constantly? That's exactly what
67 poll() solves.
68
69 PART 2 – WHAT IS poll()
70 =====
71 poll() is a system call that takes an array of {fd, events_to_watch_for}
72 and BLOCKS (efficiently, at the OS level – no CPU burned) until at least
73 one of those fds is "ready" for the requested operation. It returns the
74 same array, now with a "revents" field per entry telling you what
75 actually happened.
76
77     int poll(struct pollfd *fds, nfds_t nfd, int timeout);
78
79     struct pollfd {
80         int    fd;        // which descriptor to watch
81         short  events;    // what you're interested in (POLLIN, POLLOUT)
82         short  revents;    // what actually happened (filled in by poll())
83     };
84
85 Common flags:
86     POLLIN  -> there is data to read (or, for the listening socket, a
87                new connection is waiting to be accept()-ed)
88     POLLOUT -> the fd is writable (send() won't block)
89     POLLHUP -> the peer closed their end (hang-up)
90     POLLERR -> an error occurred on this fd
91
92 timeout = -1 means "block forever until something happens" – that's
93 what run_serv() uses, since the server has nothing else to do while
94 idle:
95
96     int ret = poll(&poll_fds[0], poll_fds.size(), -1);

```

```

97
98 WHY ONE poll() CALL FOR EVERYONE
99 -----
100 Instead of looping over every client and calling non-blocking recv() on
101 each (wasteful – most will return "nothing to read" most of the time),
102 you register ALL fds (the listening socket + every client) in one
103 poll_fds vector, and make ONE poll() call per loop iteration. The OS
104 tells you exactly which fds are actually ready, so you only call
105 recv()/send() on fds that will actually succeed immediately. This is the
106 whole point of "I/O multiplexing" and it's the #1 concept evaluators
107 will ask you to explain.
108
109 THE EVENT LOOP, STEP BY STEP (run_serv in server.cpp)
110 -----
111
112     while (true)
113     {
114         poll(&poll_fds[0], poll_fds.size(), -1);    // wait for something
115
116         for each pollfd entry:
117             if it's the LISTENING socket and revents has POLLIN:
118                 accept_new_client();    // new client connected
119
120             else if revents has (POLLHUP | POLLERR):
121                 disconnect_client();    // client dropped / errored
122
123             else:
124                 if revents has POLLIN:
125                     lire_depuis_client(); // read + parse + dispatch
126                 if revents has POLLOUT:
127                     send_repons_to_client(); // flush queued output
128     }
129
130 Notice client fds start out watching ONLY POLLIN (see accept_new_client:
131 client_poll.events = POLLIN). POLLOUT is added dynamically, only when
132 there's something queued to send (queue_response does
133 poll_fds[i].events |= POLLOUT), and removed again once the output buffer
134 is empty (send_repons_to_client does events &= ~POLLOUT). This matters:
135 if you always watched POLLOUT, poll() would return immediately on every
136 call (since a socket is ALWAYS writable when idle), spinning your CPU at
137 100%. Watching POLLOUT only when you actually have pending data is the
138 correct pattern – and evaluators specifically check for this.
139
140 WHY THE index BOOKKEEPING IS TRICKY
141 -----
142 poll_fds and clients (a map<fd, Client>) must stay in sync. When a
143 client disconnects mid-loop, disconnect_client() erases their entry from
144 poll_fds, which SHIFTS every following index down by one. That's why

```

145 run_serv uses `continue` instead of `i++` right after any function that
146 might disconnect a client (disconnect_client itself, or a failed
147 read/write) – otherwise the next iteration would skip a client entirely,
148 or read reverts from the wrong/now-invalid pollfd slot. This is a
149 classic bug evaluators try to trigger by disconnecting a client abruptly
150 (closing nc, killing a terminal) mid-conversation.

151
152 PART 3 – WHAT THE FT_IRC EVALUATION USUALLY CHECKS

153 =====
154 This project is graded live, typically against the 42 "ft_irc" subject.
155 Below is what you should be ready to explain and demonstrate.

- 156
157 1. NO BLOCKING, EVER
- 158 - Every fd must be non-blocking. If you accidentally block on ONE
159 client, the WHOLE server freezes for ALL clients – evaluators test
160 this by having 2+ terminals connected and checking one client
161 hanging doesn't freeze another's chat.
 - 162 - You must use exactly ONE poll() (or equivalent) covering ALL fds –
163 not one recv() per client in a loop, not multiple poll() calls.
- 164
165 2. PARTIAL READS/Writes ARE HANDLED CORRECTLY
- 166 - A single recv() may return a partial line, multiple lines, or a
167 line split across two recv() calls. Demonstrated here by
168 buffer/extract_line in client.cpp.
 - 169 - A single send() may not send everything at once – demonstrated by
170 outp_Buffer/consume_outp + only clearing POLLOUT once the queue is
171 actually empty.
 - 172 - Test this yourself by pasting a huge block of text into nc, or by
173 sending a command byte-by-byte with a slow script – the server must
174 not crash or corrupt data.
- 175
176 3. THE SERVER NEVER CRASHES OR QUITs UNEXPECTEDLY
- 177 - No segfaults, no unhandled exceptions, no exit() on bad client
178 input. A malformed command should get an error numeric reply, not
179 kill the server.
 - 180 - Test with garbage input: raw bytes, huge lines, empty lines, lines
181 with only "\r\n", commands with wrong number of params, non-UTF8
182 bytes.
 - 183 - Ctrl+C – check for clean shutdown behavior (or at least no crash);
184 confirm whether your subject requires a signal handler.
- 185
186 4. REGISTRATION FLOW (PASS / NICK / USER) MATCHES THE SUBJECT
- 187 - Client must authenticate with the correct password before doing
188 anything else. Here: handl_pass sets pass_ok, and try_register only
189 flips registered=true once pass_ok && nick set && user set – and
190 process_command rejects any other command with 451 until then. Be
191 ready to explain that exact gate.
 - 192 - Be ready to explain the ORDER doesn't matter (NICK before or after

USER, etc.) since `try_register` is called from all three handlers and just checks the three conditions independently.

5. MULTI-CLIENT / MULTI-CHANNEL BEHAVIOR

- JOIN, PART, PRIVMSG (to a channel and to a nick), broadcast correctly to all members except (in some cases) the sender – check `diffusion_msg_to_channel`'s `exclude_fd` parameter usage carefully: JOIN broadcasts to everyone including the joiner (-1 = exclude nobody) but PRIVMSG excludes the sender (fd) so they don't see their own message echoed back.
- Operators vs regular members – first joiner becomes operator automatically (see `handl_join`).
- KICK/INVITE/TOPIC access control (operator-only where required).
- Test manually with at least 2-3 concurrent ``nc localhost <port>`` or an actual IRC client (irssi, WeeChat, HexChat) – evaluators often use a real client, which is pickier about exact reply formatting than nc.

6. NUMERIC REPLIES MATCH THE IRC FORMAT

- Format is `":servername CODE target :text\r\n"` – check `send_numeric_reply/format_code` produce exactly that (3-digit code, always). Common codes you already have: 1 (welcome – should really be 001, double check your subject's expectations), 401, 403, 411, 412, 421, 431, 433, 441, 442, 443, 451, 461, 462, 464, 482.
- Missing ones you'll likely need to add for MODE: 324 (channel modes), 472 (unknown mode char), 473 (invite only), 475 (bad channel key), 471 (channel full).

7. NO FORBIDDEN FUNCTIONS / STANDARDS COMPLIANCE

- C++98 only (see Makefile: `-std=c++98`) – no auto, no range-for, no nullptr, no `<thread>`. Don't accidentally use C++11 features while adding MODE.
- `-Wall -Wextra -Werror` : the build must be 100% warning-free.
- Typically forbidden: `fork()` for handling clients (threads/forking per connection defeats the purpose of this project – one process, one `poll()` loop only).
- Usually mandatory: exactly one `poll()`-family call (`poll/select/epoll/kqueue`) serving read AND write for ALL fds – check your specific subject text for which multiplexer is required.

8. MEMORY / RESOURCE MANAGEMENT

- Every `accept()`-ed fd must eventually be `close()`-d (see `disconnect_client` and the Server destructor looping over `poll_fds`).
- No leaks – run under `valgrind` while connecting/disconnecting clients repeatedly; check no growing memory over time.
- Channels are erased when empty (see `is_empty()` checks after PART/KICK/disconnect) so you don't leak empty Channel objects forever in the map.

9. BE READY TO EXPLAIN, LIVE, IN YOUR OWN WORDS

- "Walk me through what happens from the moment a client connects to the moment they send their first message." (accept_new_client -> poll_fds entry -> client sends bytes -> POLLIN -> recv -> buffer -> extract_line -> parse_message -> process_command -> handler -> queue_response -> POLLOUT -> send)
- "What happens if two clients try to take the same nickname at the same time?" (is_nick_taken check inside handl_nick; no true race condition since it's single-threaded – whichever NICK command is processed first, by poll() ordering, wins)
- "Why non-blocking sockets? Why poll()? What would go wrong without them?"
- "What happens if send() only sends half the buffer?" (consume_outp erases only the sent portion; POLLOUT stays set until empty)
- "Show me it doesn't crash on bad input" (be ready to type garbage into nc live)

PART 4 – QUICK TESTING CHECKLIST BEFORE YOUR EVAL

=====

```
make re                                # rebuild clean, check for
                                       # warnings (-Werror means
                                       # ANY warning = build fails)
```

```
./ircserv 6667 pass123
```

```
# basic connection test:
```

```
nc -C localhost 6667
```

```
PASS pass123
```

```
NICK alice
```

```
USER alice 0 * :Alice
```

```
# open a 2nd terminal as "bob", same steps, then:
```

```
JOIN #test          (in both)
```

```
PRIVMSG #test :hi   (in one, confirm the other receives it)
```

```
TOPIC #test :new topic
```

```
KICK #test bob :bye
```

```
INVITE bob #test
```

```
PART #test
```

```
# abuse tests:
```

```
- send an empty line
```

```
- send a line with no \n (leave connection idle) then send more later
  -> confirm it's buffered correctly, not dropped/duplicated
```

```
- send a huge single line (several KB) -> confirm no crash
```

```
- kill one nc client abruptly (Ctrl+C) mid-JOIN -> confirm other
  clients + server keep working
```

```
- connect with WRONG password -> confirm rejected, no crash
```

```
289 | - send commands before registering -> confirm 451, not a crash
290 | - run under valgrind --leak-check=full, repeat the above, check for
291 |   leaks after clients disconnect
292 |
293 | # real client test (evaluators often do this):
294 | irssi -c localhost -p 6667 (or /connect / /server in HexChat, etc.)
295 | # real clients are stricter about numeric reply formatting than nc,
296 | # so if something looks weird here, fix the numeric reply first.
297 |
298 | If everything above passes cleanly, you're in good shape for the
299 | mandatory part of the evaluation. MODE (see the other file,
300 | filt_irc.txt) is the main remaining functional gap in this codebase.
301 |
```